CV INTELLIGENCE MATCHER

using RAG + LLM



An AI-powered CV Intelligence Matcher system that leverages Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs) to analyze resumes and answer natural language questions intelligently.

2026

Abstract

CV Intelligence Matcher using RAG + LLM

This project presents an AI-powered CV Intelligence Matcher system using Retrieval-Augmented Generation (RAG) and Large Language Models (LLMs).
This system allows users to upload PDF resumes and ask natural language questions about candidates.
The application extracts information from resumes, stores embeddings in a vector database, retrieves relevant content,
and generates accurate answers using a local LLM through Ollama.

The system supports both English and Arabic queries and provides a modern user-friendly interface
in addition to developer API documentation.

1. Introduction

Recruitment systems often require manual review of large numbers of resumes, which consumes significant time and effort.
Traditional keyword-based search systems may fail to understand semantic meaning and contextual relationships
between skills and experiences.

To solve this problem, we developed a CV Intelligence Matcher system based on Retrieval-Augmented Generation (RAG).
The system combines semantic search with Large Language Models to provide intelligent resume analysis and question answering.

The application enables recruiters to:

- Upload PDF resumes
- Retrieve relevant candidates based on skills and experience
- Work completely offline using local LLMs
- Ask questions using natural language
- Analyze resumes using AI
- Provide accurate and context-aware results

2. Objectives

The main objectives of the project are:

- [x] Build a smart resume analysis system
- [x] Implement Retrieval-Augmented Generation (RAG)
- [x] Support semantic search instead of keyword search
- [x] Integrate local LLMs using Ollama
- [x] Store embeddings using Qdrant vector database
- [x] Create a modern and user-friendly interface
- [x] Support English and Arabic queries
- [x] Run completely offline after setup

4. System Architecture

The system workflow consists of the following steps:

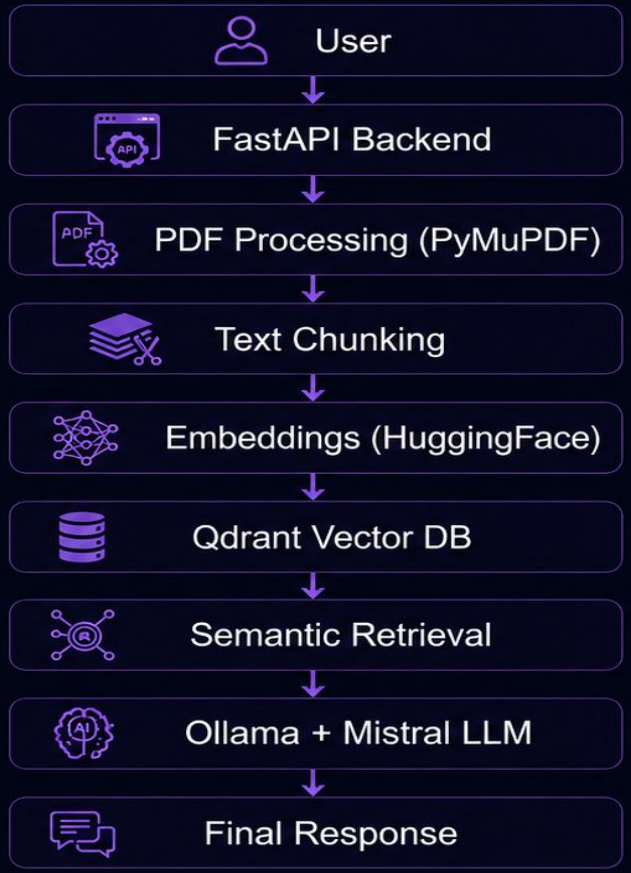
1. User uploads a PDF resume
2. PDF text is extracted using PyMuPDF
3. Text is split into chunks
4. Embeddings are generated using HuggingFace model
5. Embeddings are stored in Qdrant vector database
6. User asks a question
7. Semantic search retrieves relevant chunks
8. Retrieved context is sent to the LLM
9. LLM generates the final answer

3. Technologies Used :

Technology	Purpose
FastAPI	Backend API
LangChain	RAG Pipeline
Ollama	Local LLM Inference
Mistral	Language Model
Qdrant	Vector Database
HuggingFace Embeddings	Text Embedding
Docker	Containerization
HTML/CSS/JavaScript	User Interface
PyMuPDF	PDF Processing

4. System Workflow

User Upload
↓
FastAPI Backend
↓
PDF Processing
↓
Text Chunking (PyMuPDF)
↓
Embeddings (HuggingFace)
↓
Qdrant Vector DB
↓
Semantic Retrieval
↓
Final Response



5. Backend Implementation

FastAPI Backend

FastAPI was used to build REST APIs for handling file uploads, document processing, semantic search, and question answering.

Main Endpoints

Endpoint	Description
/upload	Upload PDF resumes
/process/{file_name}	Process and index PDF
/search	Semantic similarity search
/answer	RAG-based question answering
/ui	User interface
/docs	Swagger API documentation



6. Vector Database and Embeddings

The system uses HuggingFace sentence-transformer embeddings to convert text into vector representations.

Model used:
all-MiniLM-L6-v2

The embeddings are stored in Qdrant vector database using cosine similarity. This allows semantic retrieval based on meaning rather than exact keywords.

7. Retrieval-Augmented Generation (RAG)

RAG combines retrieval systems with language models.

The retrieval step fetches the most relevant resume chunks from the vector database. These chunks are then provided to the language model as context.

The LLM generates answers based only on the retrieved information.

To reduce hallucination:

- [x] Strict prompting was used
- [x] Similarity filtering was applied
- [x] Context-only answering was enforced

8. Ollama and Local LLM Integration

The system integrates Ollama to run local LLMs completely offline.

Model used:
Mistral

Benefits:

- Offline execution
- Better privacy
- No API cost
- Faster local inference

Communication with Ollama was implemented using HTTP requests from FastAPI.

DOCKER USAGE

Why Docker was used in this project

CV MATCHER
AI-Powered CV Matching System

1. Consistency

Docker ensures the application runs the same way in development, testing, and production environments.

2. Isolation

All dependencies, libraries, and runtime environments are packaged inside containers, avoiding conflicts with other systems.

3. Portability

The entire application can be easily moved and deployed on any system that has Docker installed.

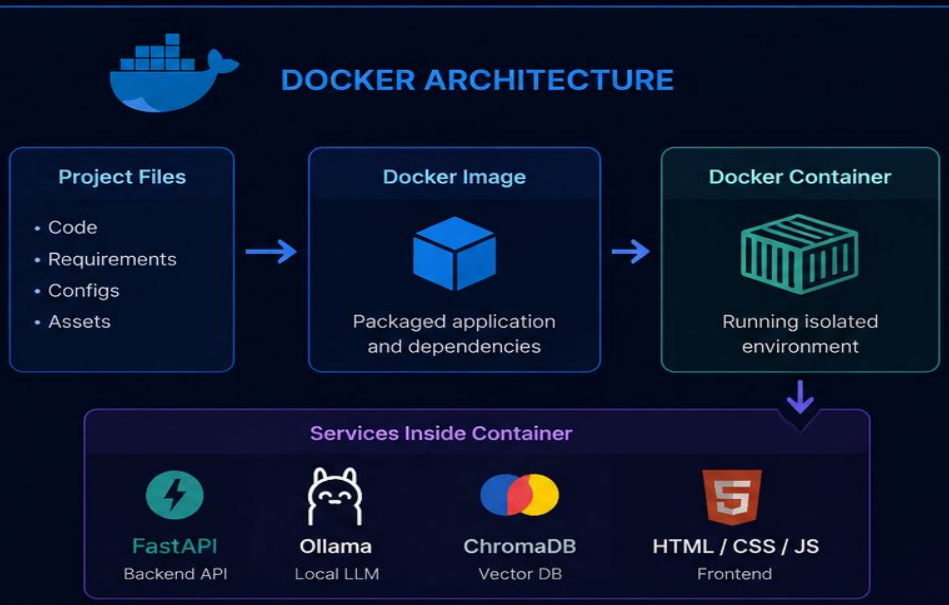
4. Easy Deployment

Docker simplifies setup and deployment with a single command using `docker compose`.

IMPACT

Docker made the development workflow faster, deployment smoother, and the system more reliable and scalable.

DOCKER ARCHITECTURE



Deployment Command

```
docker compose up --build -d
```

This builds the image and runs the container in the background.

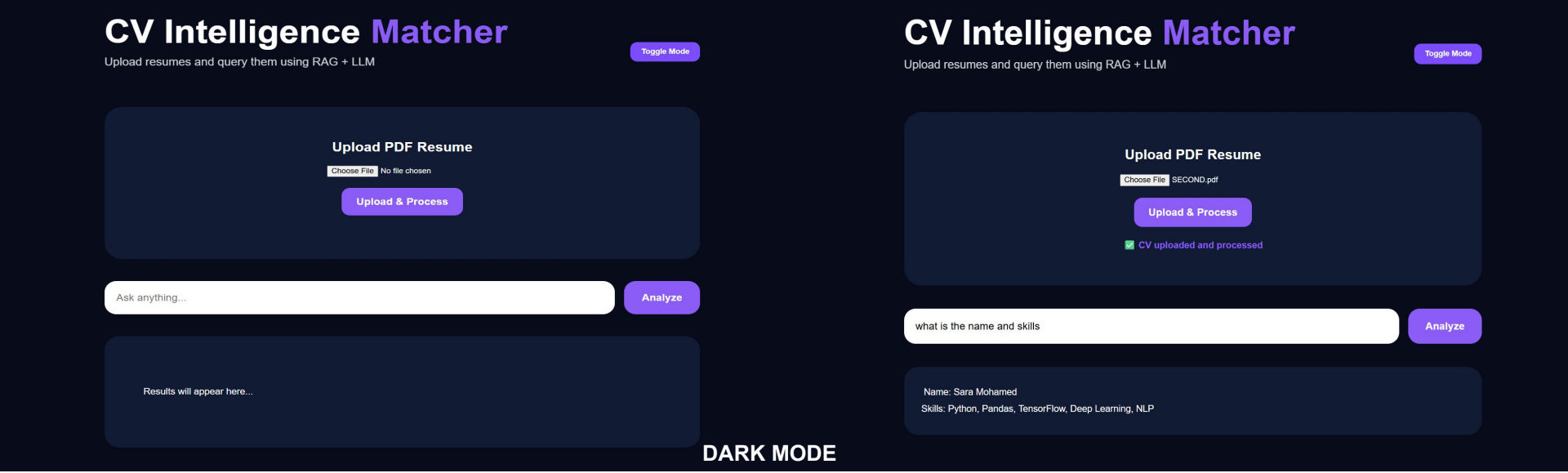
9. User Interface

A modern UI was designed for normal users.



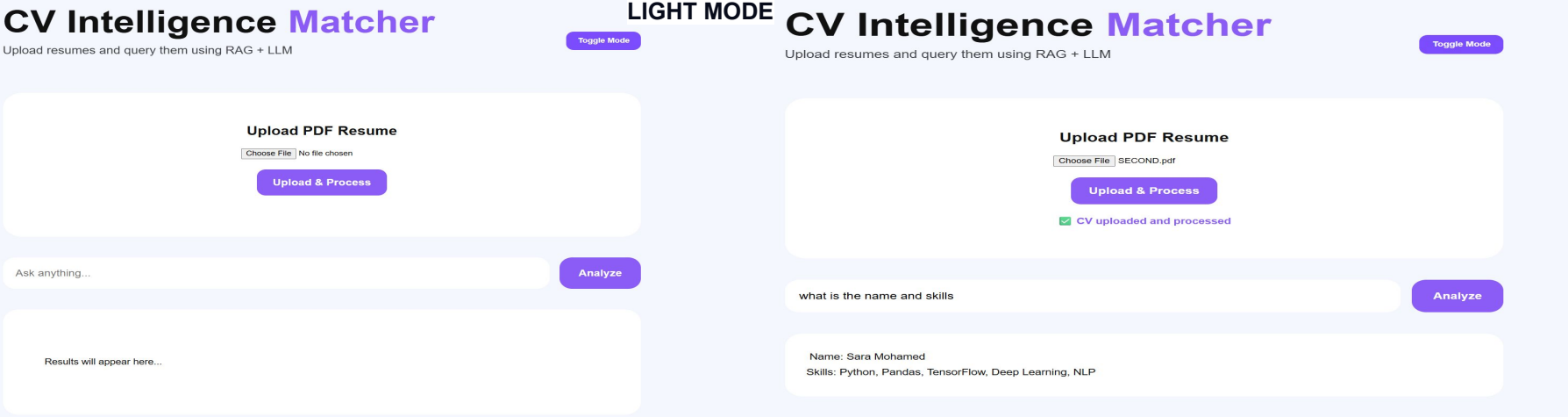
Features:

- Upload PDF resumes
- Ask questions naturally
- Dark/Light mode toggle
- Clean modern design
- Real-time responses
- Easy interaction



DARK MODE

LIGHT MODE



10. Testing and Results

The system was tested using multiple English and Arabic resumes.

Example queries:

- Who has Docker skills?
- Who worked on AI projects?
- Does Michael Brown know Linux?
- من لديه خبرة في التعلم العميق؟ ⟷ > (arabic support)

The system successfully retrieved relevant candidates and generated accurate responses.

Hallucination in LLM responses :

The model sometimes generated incorrect answers not found in the provided documents.

11. Challenges Faced

Several challenges were encountered during development:

- Hallucination in LLM responses
- Incorrect semantic retrieval
- Docker dependency installation issues
- UI integration problems
- Chunk splitting accuracy
- Arabic query support

Solutions included:

- Strict prompting
- Similarity filtering
- Better chunk sizes
- Collection reset mechanism
- Improved retrieval pipeline

Cv example to test :

Name: Sara Mohamed

Skills : Python, Pandas, TensorFlow, Deep Learning, NLP

Experience:

- Data Scientist at Orange (3 years)
- ML Engineer at Valeo (2 years)

Education:

MSc Artificial Intelligence

16. Hallucination in LLM Responses

The system is designed to minimize hallucinations by relying only on the retrieved context.
If the information is not available in the context, the model clearly states that it cannot be found.

01

is she worked at microsoft?

Analyze

I could not find any information in the provided context that Sara Mohamed has worked at Microsoft.

Not Found in Context

02

is she worked at Orange?

Analyze

Yes, she was a Data Scientist at Orange for 3 years.

Found in Context

Why this matters?

By preventing hallucinations, we ensure that users get accurate, trustworthy, and context-grounded answers.

- Accurate Answers
- Reliable System
- User Trust

Hallucination in LLM responses :

is she worked at microsoft?

Analyze

I could not find any information in the provided context that Sara Mohamed has worked at Microsoft.

is she worked at Orange?

Analyze

Yes, she was a Data Scientist at Orange for 3 years.

Note : Screenshots from our project

17. Arabic Language Support

دعم اللغة العربية

The system fully supports Arabic queries and provides accurate, context-based answers.

سؤال بالعربية
Arabic Question

Analyze

هل تجيد الذكاء الصناعي

الإجابة
Answer

نعم، سارة محمد لديها خبرة في هندسة تعلم الآلة.

Yes, Sara Mohamed has experience in Machine Learning Engineering.

سؤال بالعربية
Arabic Question

Analyze

هل تجيد الشبكات؟

الإجابة
Answer

لا أستطيع العثور على معلومات في المستندات تفيد بأن سارة محمد متخصصة في إنشاء الشبكات العصبية.

I could not find information in the documents that Sara Mohamed specializes in creating neural networks.

Providing Arabic support ensures a better experience for users and enables wider accessibility.

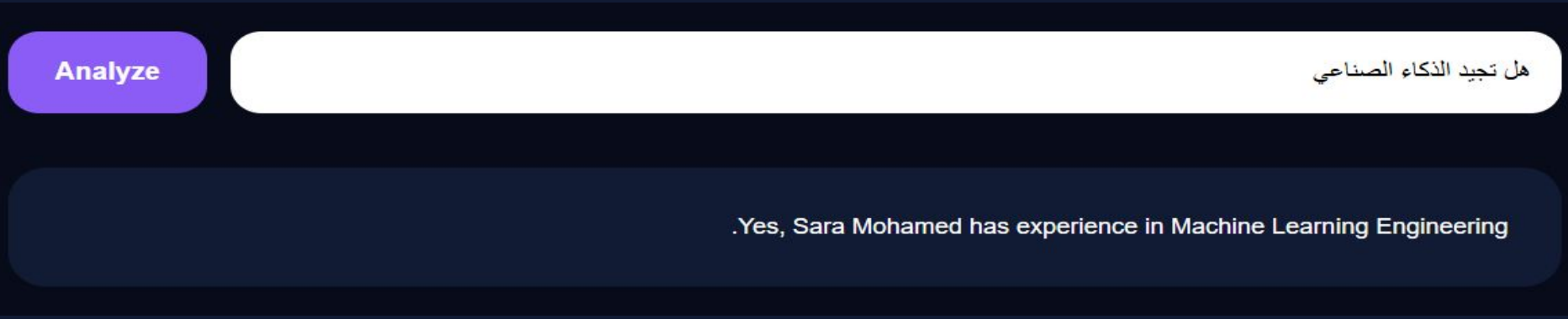
Natural Arabic Queries

Accurate Answers

Better User Experience

Arabic support :

دعم اللغه العربيه:



Note : Screenshots from our project

CV Intelligence Matcher

Upload resumes and query them using RAG + LLM

Toggle Mode

Upload PDF Resume

Choose File CV4.pdf

Upload & Process

CV uploaded and processed

Ask anything...

Analyze

Results will appear here...

NOTE : NORMAL USER INTERFACE

12. Future Improvements



Possible future enhancements include:

- Multi-user authentication
- Cloud deployment
- Better reranking systems
- Advanced filtering and ranking
- OCR support for scanned PDFs
- Multi-file simultaneous indexing
- Candidate scoring system
- Chat history support

13. Conclusion :



The project successfully demonstrates the implementation of a Retrieval-Augmented Generation system for intelligent resume analysis.

The application combines semantic search, vector databases, and local LLMs to provide accurate and context-aware responses.

The final system is modern, scalable, offline-capable, and suitable for AI-powered recruitment assistance.

14. Developer API Interface

In addition to the modern user interface designed for normal users, the system also provides a developer-oriented API interface using Swagger UI through FastAPI.

This interface allows developers and testers to directly interact with backend endpoints, test API requests, upload files, process documents, and retrieve answers without using the graphical UI.

The developer interface is useful for:

- API testing
- Backend debugging
- Endpoint validation
- Manual request simulation
- Development and integration purposes

Available API Endpoints

Endpoint	Function
/upload	Upload PDF resumes
/process/{file_name}	Process and index PDF
/search	Perform semantic similarity search
/answer	Generate AI answers using RAG
/docs	Swagger developer interface
/ui	Modern user interface

How to Use the Developer Interface

1 Open Swagger UI

Open the following URL:

`http://localhost:8000/docs`

2 Upload PDF

Use the `/upload` endpoint.

- Click Try it out
- Choose a PDF file
- Click Execute

The system uploads the resume successfully.

3 Process the PDF

Use: `/process/{file_name}`

Example: `/process/cv.pdf`

This step:

- extracts text
- creates embeddings
- stores vectors in Qdrant

4 Ask Questions

Use the `/answer` endpoint.

Example queries:

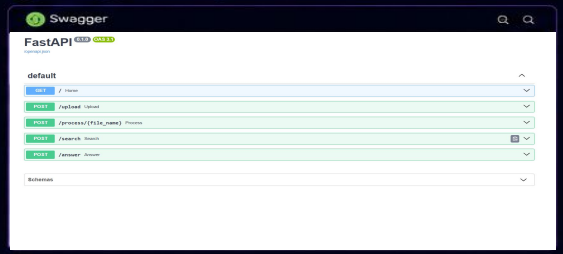
`Who has Docker skills?`

`Who worked on AI projects?`

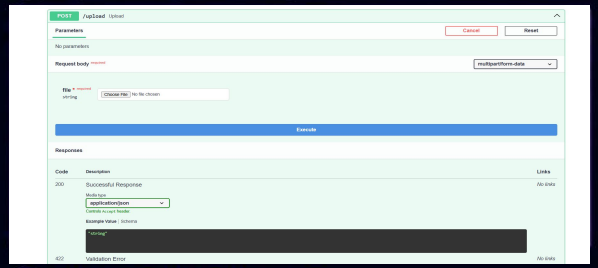
`من لديه خبرة في التعلم العميق؟`

The system retrieves relevant resume information and generates an AI-based response.

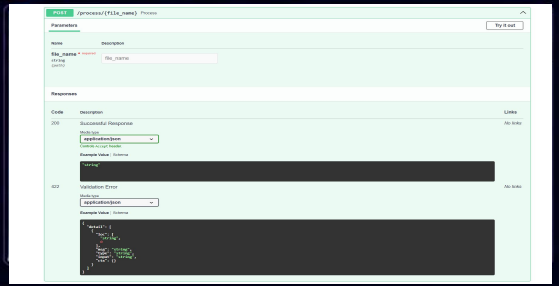
1 Swagger Developer Interface



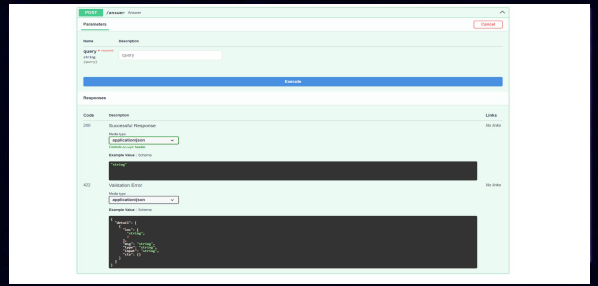
2 Upload Endpoint Example



3 Process Endpoint Example



4 Answer Endpoint Example



Suggested Screenshot Placement

1. **Swagger Developer Interface** — Take screenshot from: `/docs`
2. **Upload Endpoint Example** — Show successful PDF upload.
3. **Process Endpoint Example** — Show successful PDF processing.
4. **Answer Endpoint Example** — Show AI-generated answer from Swagger UI.

15. From Developer Interface to User-Friendly Experience

The initial Swagger-based developer interface was functional for API testing and backend development, but it was not practical for normal users. The interface appeared technical, complex, and difficult for non-developers to interact with. Users were required to manually execute API endpoints, upload files through developer tools, and understand backend operations, which reduced usability and overall user experience.

To solve this issue, a modern user-friendly interface was designed with a cleaner layout, simplified interactions, dark/light mode support, and direct AI querying functionality suitable for regular users.

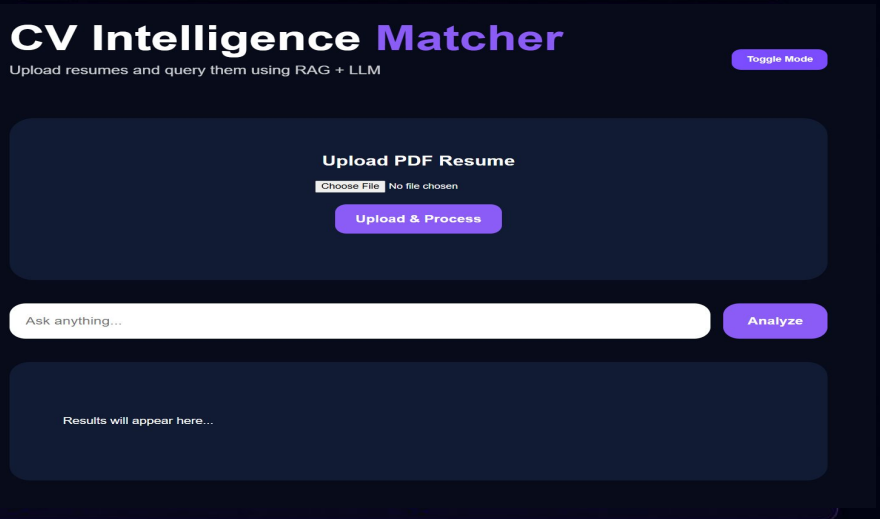
Our Goal

- Make the system easy to use for everyone
- Remove technical complexity
- Provide a clean and modern experience
- Enable natural interaction with AI

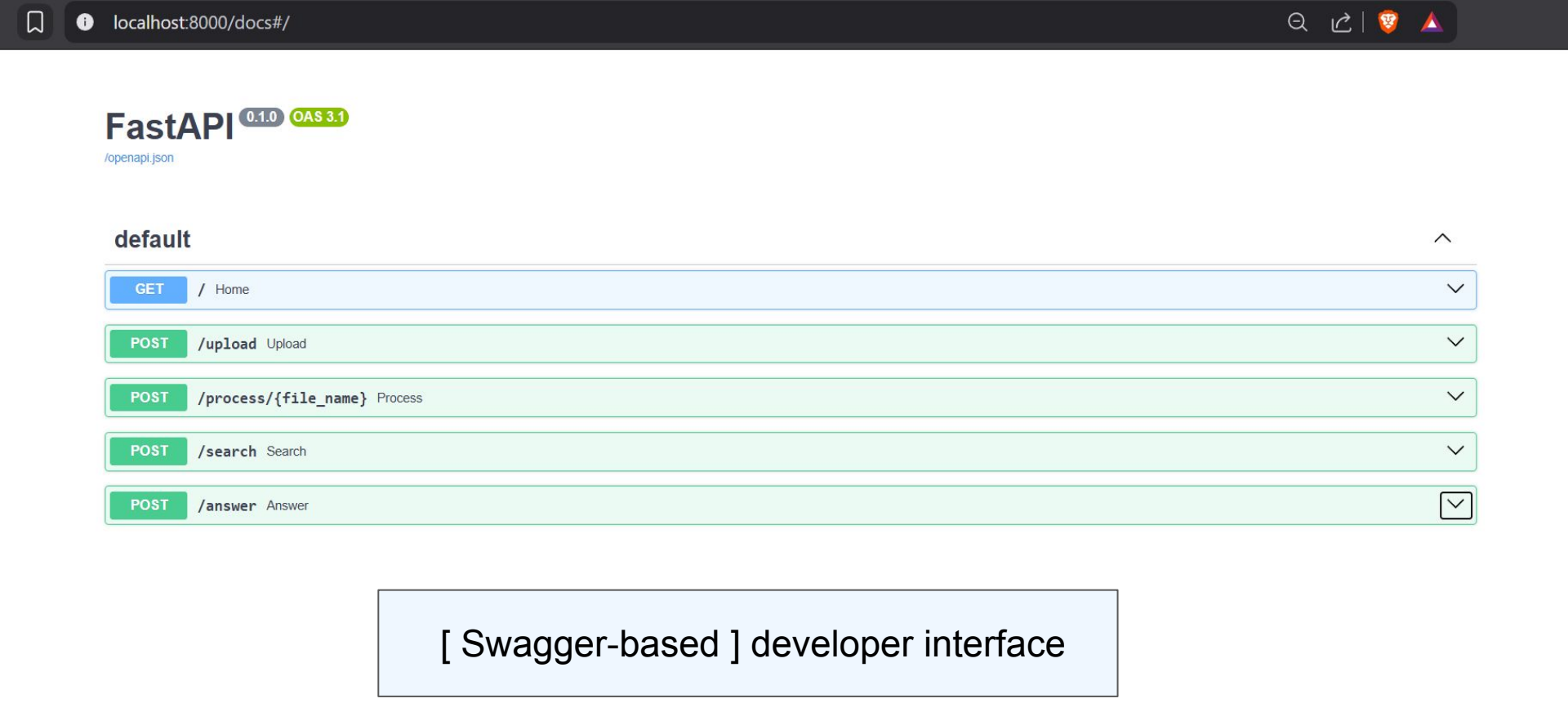
BEFORE: Developer Interface (Swagger)



- Technical and complex layout
- Not suitable for normal users
- Requires knowledge of APIs
- Manual endpoint execution
- Poor user experience
- Not intuitive



The new interface makes the system **accessible, efficient,** and **enjoyable** for all users, not just developers.



[Swagger-based] developer interface

default

GET / Home

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8000/
```

Server response

Code	Details
200	Response body

```
{
  "message": "RAG WORKING"
}
```

Download

Response headers

```
content-length: 25
content-type: application/json
date: Fri,08 May 2026 11:03:52 GMT
server: uvicorn
```

POST /upload Upload

Parameters

Cancel Reset

No parameters

Request body required

multipart/form-data

file * required
string

Choose File SECOND.pdf

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/upload' \
  -H 'accept: application/json' \
  -H 'Content-Type: multipart/form-data' \
  -F 'file=@SECOND.pdf;type=application/pdf'
```

Request URL

```
http://localhost:8000/upload
```

Server response

Code	Details
200	Response body

```
{
  "message": "uploaded successfully",
  "file": "SECOND.pdf"
}
```

Download

Response headers

```
content-length: 55
content-type: application/json
date: Fri,08 May 2026 11:04:46 GMT
server: uvicorn
```

POST /process/{file_name} Process

Parameters Cancel

Name	Description
file_name * required string *(path)*	SECOND.pdf

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/process/SECOND.pdf' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8000/process/SECOND.pdf
```

Server response

Code	Details
200	Response body

```
{
  "chunks_count": 1,
  "message": "stored in qdrant"
}
```

Download

Response headers

```
content-length: 47
content-type: application/json
date: Fri,08 May 2026 11:05:46 GMT
server: uvicorn
```

POST /answer Answer

Parameters

Cancel

Name	Description
query * required string *(query)*	what is the name only

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/answer?query=what%20is%20the%20name%20only' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8000/answer?query=what%20is%20the%20name%20only
```

Server response

Code	Details
200	Response body

```
{
  "query": "what is the name only",
  "answer": " Sara Mohamed"
}
```

Download

Response headers

```
content-length: 58
content-type: application/json
date: Fri,08 May 2026 11:07:58 GMT
server: uvicorn
```

POST /answer Answer

Parameters

Cancel

Name	Description
query * required string *(query)*	what is the name and skills

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/answer?query=what%20is%20the%20name%20and%20skills' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8000/answer?query=what%20is%20the%20name%20and%20skills
```

Server response

Code	Details
200	Response body

```
{
  "query": "what is the name and skills",
  "answer": " Name: Sara Mohamed\nSkills: Python, Pandas, TensorFlow, Deep Learning, NLP"
}
```

Download

Response headers

```
content-length: 126
content-type: application/json
date: Fri,08 May 2026 11:09:07 GMT
server: uvicorn
```

POST /answer Answer

Parameters

Cancel

Name	Description
query * required string *(query)*	is sara mohamed have networking skills?

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/answer?query=is%20sara%20mohamed%20have%20networking%20skills%3F' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8000/answer?query=is%20sara%20mohamed%20have%20networking%20skills%3F
```

Server response

Code	Details
200	Response body

```
{
  "query": "is sara mohamed have networking skills?",
  "answer": " I could not find the answer in the documents regarding Sara Mohamed having networking skills."
}
```

Download

Response headers

```
content-length: 157
content-type: application/json
date: Fri,08 May 2026 11:10:33 GMT
server: uvicorn
```

POST /answer Answer

Parameters

Cancel

Name	Description
query * required string *(query)*	هل تجيد بايثون؟

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/answer?query=%D9%87%D9%84%20%D8%AA%D8%AC%D9%8A%D8%AF%20%D8%A8%D8%A7%D9%8A%D8%AB%D9%88%D9%86%D8%9F' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8000/answer?query=%D9%87%D9%84%20%D8%AA%D8%AC%D9%8A%D8%AF%20%D8%A8%D8%A7%D9%8A%D8%AB%D9%88%D9%86%D8%9F
```

Server response

Code	Details
200	Response body

```
{
  "query": "هل تجيد بايثون؟",
  "answer": " Yes, Sara Mohamed could find Python."
}
```

Download

Response headers

```
content-length: 89
content-type: application/json
date: Fri,08 May 2026 11:13:02 GMT
server: uvicorn
```

POST /answer Answer

Parameters

Cancel

Name	Description
query * required string *(query)*	هل عملت في مايكروسفت؟

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/answer?query=%D9%87%D9%84%20%D8%B9%D9%85%D9%84%D8%AA%20%D9%81%D9%8A%20%D9%85%D8%A7%D9%8A%D9%83%D8%B1%D9%88%D8%B3%D9%81%D8%AA%D8%9F' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8000/answer?query=%D9%87%D9%84%20%D8%B9%D9%85%D9%84%D8%AA%20%D9%81%D9%8A%20%D9%85%D8%A7%D9%8A%D9%83%D8%B1%D9%88%D8%B3%D9%81%D8%AA%D8%9F
```

Server response

Code	Details
200	Response body

```
{
  "query": "هل عملت في مايكروسفت؟",
  "answer": " I could not find that Sara Mohamed has worked for Microsoft in the provided context."
}
```

Download

Response headers

```
content-length: 148
content-type: application/json
date: Fri,08 May 2026 11:14:01 GMT
server: uvicorn
```

POST /answer Answer

Parameters

Cancel

Name	Description
query * required string *(query)*	هل عملت في اورانج؟

Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/answer?query=%D9%87%D9%84%20%D8%B9%D9%85%D9%84%D8%AA%20%D9%81%D9%8A%20%D8%A7%D9%88%D8%B1%D8%A7%D9%86%D8%AC%D8%9F' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

```
http://localhost:8000/answer?query=%D9%87%D9%84%20%D8%B9%D9%85%D9%84%D8%AA%20%D9%81%D9%8A%20%D8%A7%D9%88%D8%B1%D8%A7%D9%86%D8%AC%D8%9F
```

Server response

Code	Details
200	Response body

```
{
  "query": "هل عملت في اورانج؟",
  "answer": " Yes, she worked as a Data Scientist at Orange for three years."
}
```

Download

Response headers

```
content-length: 120
content-type: application/json
date: Fri,08 May 2026 11:14:49 GMT
server: uvicorn
```